

```
29 public class Solution {
30     public static void main(String[] args) throws IOException {
31         BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
32         BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv(
33             "OUTPUT_PATH")));
34
35         String[] firstMultipleInput = bufferedReader.readLine().replaceAll("\\s+$", "").split("
36
37         int month = Integer.parseInt(firstMultipleInput[0]);
38
39         int day = Integer.parseInt(firstMultipleInput[1]);
40
41         int year = Integer.parseInt(firstMultipleInput[2]);
42
43         String res = Result.findDay(month, day, year);
44
45         bufferedWriter.write(res);
46         bufferedWriter.newLine();
47
48         bufferedReader.close();
49         bufferedWriter.close();
50     }
51 }
```

1154. Day of the Year

Easy Topics Companies Hint

Given a string `date` representing a [Gregorian calendar](#) date formatted as `YYYY-MM-DD`, return *the day number of the year*.

Example 1:

Input: `date = "2019-01-09"`
Output: 9
Explanation: Given date is the 9th day of the year in 2019.

Example 2:

Input: `date = "2019-02-10"`
Output: 41

Constraints:

- `date.length == 10`
- `date[4] == date[7] == '-'`, and all other `date[i]`'s are digits
- `date` represents a calendar date between Jan 1st, 1900 and Dec 31st, 2019.



Seen this question in a real interview before? 1/6

Yes No

Accepted 102,396/203.8K Acceptance Rate 50.2%

518 36

12 Online

Code

Java Auto

```
1 class Solution {
2     public int dayOfYear(String date) {
3         int year = Integer.parseInt(date.substring(0, 4));
4         int month = Integer.parseInt(date.substring(5, 7));
5         int day = Integer.parseInt(date.substring(8));
6
7         int[] days = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
8
9         // Check for leap year
10        boolean isLeap = (year % 400 == 0) || (year % 4 == 0 && year % 100 != 0);
11        if (isLeap && month > 2) {
12            day += 1;
13        }
14
15        // Add days of previous months
16        for (int i = 0; i < month - 1; i++) {
17            day += days[i];
18        }
19    }
20 }
```

Saved

Ln 23, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`date = "2019-01-09"`

Output

9

Expected

9

Contribute a testcase

1360. Number of Days Between Two Dates

Easy Topics Companies Hint

Write a program to count the number of days between two dates.

The two dates are given as strings, their format is YYYY-MM-DD as shown in the examples.

Example 1:

Input: date1 = "2019-06-29", date2 = "2019-06-30"

Output: 1

Example 2:

Input: date1 = "2020-01-15", date2 = "2019-12-31"

Output: 15

Constraints:

- The given dates are valid dates between the years 1971 and 2100.

UNLOCK EXPERIENCES

Prepare your network for
today and tomorrow

HPE

Seen this question in a real interview before? 1/6

Yes No

Accepted 77,896/146.2K Acceptance Rate 53.3%

Topics

443 42



Hot weather
Now



16 Online



Java Auto

```
1 import java.time.LocalDate;
2 import java.time.temporal.ChronoUnit;
3
4 class Solution {
5     public int daysBetweenDates(String date1, String date2) {
6         LocalDate d1 = LocalDate.parse(date1);
7         LocalDate d2 = LocalDate.parse(date2);
8         return (int) Math.abs(ChronoUnit.DAYS.between(d1, d2));
9     }
10 }
11
```

Saved

Ln 11, Col 1

Testcase Test Result

Accepted Runtime: 6 ms

Case 1

Case 2

Input

date1 =
"2019-06-29"

date2 =
"2019-06-30"

Output

1

Expected

1

Copilot

ENG

IN

11:22
14-08-2026

Description Editorial Solutions Submissions

1185. Day of the Week

Easy Topics Companies Hint

Given a date, return the corresponding day of the week for that date.

The input is given as three integers representing the `day`, `month` and `year` respectively.

Return the answer as one of the following values ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"].

Note: January 1, 1971 was a Friday.

Example 1:

Input: day = 31, month = 8, year = 2019
Output: "Saturday"

Example 2:

Input: day = 18, month = 7, year = 1999
Output: "Sunday"

Example 3:

Input: day = 15, month = 8, year = 1993
Output: "Sunday"

Constraints:

- The given dates are valid dates between the years 1971 and 2100.

GoDaddy

Great ideas deserve a great domain name.

📄 < > 📄 Code

```
Java Auto
1 import java.time.LocalDate;
2
3 class Solution {
4     public String dayOfTheWeek(int day, int month, int year) {
5         LocalDate date = LocalDate.of(year, month, day);
6         String weekDay = date.getDayOfWeek().name();
7         // Capitalize first letter and lowercase the rest
8         return weekDay.charAt(0) + weekDay.substring(1).toLowerCase();
9     }
10 }
11
```

Saved

Ln 11, Col 1

📄 Testcase 📄 Test Result

Accepted Runtime: 5 ms

✅ Case 1 ✅ Case 2 ✅ Case 3

Input

day =
31

month =
8

year =
2019

Output

"Saturday"

12 Online

```
17 ArrayList<ArrayList<Integer>> times = new ArrayList<>();
18
19 ✓
20 int d = scan.nextInt();
21 ArrayList<Integer> row = new ArrayList<>();
22 ✓ for (int j = 0; j < d; j++) {
23     row.add(scan.nextInt());
24 }
25
```

Line: 49 Col: 1



Upload Code as File



Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.



Sample Test case 0



Sample Test case 1

Input (stdin)

```
1 5
2 5 41 77 74 22 44
3 1 12
4 4 37 34 36 53
```

Download

```
1  import java.util.*;
2
3  class Student{
4      private int id;
5      private String fname;
6      private double cgpa;
7      public Student(int id, String fname, double cgpa) {
8          super();
9          this.id = id;
10         this.fname = fname;
11         this.cgpa = cgpa;
12     }
13     public int getId() {
14         return id;
15     }
16     public String getFname() {
17         return fname;
18     }
19     public double getCgpa() {
20         return cgpa;
21     }
22 }
23
24 //Complete the code
25 public class Solution
```

Line: 51 Col: 1

451. Sort Characters By Frequency

Medium Topics Companies

Given a string *s*, sort it in **decreasing order** based on the **frequency** of the characters. The **frequency** of a character is the number of times it appears in the string.

Return *the sorted string*. If there are multiple answers, return *any of them*.

Example 1:

Input: *s* = "tree"
Output: "eert"
Explanation: 'e' appears twice while 'r' and 't' both appear once. So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.

Example 2:

Input: *s* = "cccaaa"
Output: "aaaccc"
Explanation: Both 'c' and 'a' appear three times, so both "cccaaa" and "aaaccc" are valid answers. Note that "cacaca" is incorrect, as the same characters must be together.

Example 3:

Input: *s* = "Aabb"
Output: "bbAa"
Explanation: "bbaA" is also a valid answer, but "Aabb" is incorrect. Note that 'A' and 'a' are treated as two different characters.

Constraints:

- 1 ≤ *s*.length ≤ 5 × 10⁵
- s* consists of uppercase and lowercase English letters and digits.

9.4K 193 ☆ 📄 🗨️

76 Online

Contribute a testcase

```
Java Auto
1 class Solution {
2     public String frequencySort(String s) {
3         // Step 1: Count the frequency of each character
4         Map<Character, Integer> map = new HashMap<>();
5         for (char c : s.toCharArray()) {
6             map.put(c, map.getOrDefault(c, 0) + 1);
7         }
8
9         // Step 2: Create a list of characters from the map keys
10        List<Character> list = new ArrayList<>(map.keySet());
11
12        // Step 3: Sort characters by frequency in descending order
13        list.sort((a, b) -> map.get(b) - map.get(a));
14
15        // Step 4: Build the result string
16        StringBuilder sb = new StringBuilder();
17        for (Character c : list) {
18            sb.append(c.repeat(map.get(c)));
19        }
20        return sb.toString();
21    }
22 }
```

Testcase Test Result

Accepted Runtime: 1 ms

Case 1 Case 2 Case 3

Input

s = "tree"

Output

"eert"

Expected

"eert"

```
4 // Write your Checker class here
5
6 class Player{
7     String name;
8     int score;
9
10    Player(String name, int score){
11        this.name = name;
12        this.score = score;
13    }
14 }
15
16 class Solution {
17     public static void main(String[] args) {
18         Scanner scan = new Scanner(System.in);
19         int n = scan.nextInt();
20
21         Player[] player = new Player[n];
22         Checker checker = new Checker();
23
24         for(int i = 0; i < n; i++){
25             player[i] = new Player(scan.next(), scan.nextInt());
26         }
```

Line: 2 Col: 1

179. Largest Number

Medium Topics Companies

Given a list of non-negative integers `nums`, arrange them such that they form the largest number and return it.

Since the result may be very large, so you need to return a string instead of an integer.

Example 1:

Input: `nums = [10,2]`

Output: `"210"`

Example 2:

Input: `nums = [3,30,34,5,9]`

Output: `"9534330"`

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 10^9$

Discover more

Number Theory Problems

Seen this question in a real interview before? 1/6

Yes No

Accepted 865,103/2M Acceptance Rate 43.6%

Topics

9.6K 228 ☆ ↗ ⌚

66 Online

Code

Java Auto

```
1 import java.util.Arrays;
2
3 class Solution {
4     public String largestNumber(int[] nums) {
5         String[] arr = new String[nums.length];
6         for (int i = 0; i < nums.length; i++) {
7             arr[i] = String.valueOf(nums[i]);
8         }
9
10        // Custom sort: compare combinations (s2 + s1) vs (s1 + s2)
11        Arrays.sort(arr, (s1, s2) -> (s2 + s1).compareTo(s1 + s2));
12
13        // Edge case: If the highest number is "0", the whole array is 0
14        if (arr[0].equals("0")) {
15            return "0";
16        }
17    }
18 }
```

Saved

Ln 26, Col 1

Testcase Test Result

Accepted Runtime: 2 ms

Case 1 Case 2

Input

`nums =`
`[10,2]`

Output

`"210"`

Expected

`"210"`

Contribute a testcase

```
8  ✓
9  public class Solution {
10     private final static Scanner scan = new Scanner(System.in);
11     private final static Priorities priorities = new Priorities();
12
13     public static void main(String[] args) {
14         int totalEvents = Integer.parseInt(scan.nextLine());
15         List<String> events = new ArrayList<>();
16
17         while (totalEvents-- != 0) {
18             String event = scan.nextLine();
19             events.add(event);
20         }
21
22         List<Student> students = priorities.getStudents(events);
23
24         if (students.isEmpty()) {
25             System.out.println("EMPTY");
26         } else {
27             for (Student st: students) {
28                 System.out.println(st.getName());
29             }
30         }
31     }
```

Line: 7 Col: 1